

EXPERIMENT 6

4

Student Name: Hunar Katyal

UID: 24BDA70085

Branch: AIT-CSE

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: ADBMS

Subject Code: 24CSP-333

Experiment 6 -- To study and implement advanced SQL queries using Joins, Correlated Query's and Conditional statement.

Aim : To study and implement correlated subqueries in SQL by retrieving employees whose salaries are among the top three highest distinct salary values within their respective departments, while correctly handling duplicate salary values using aggregate functions and joins

Question :6 (i) Calculate Apple and Orange Daily Sale difference

SQL Query:

```
SELECT sale_date, SUM(CASE WHEN fruit = 'apples' THEN sold_num ELSE -sold_num END) as s
FROM Sales
GROUP BY sale_date
```

	sale_date date	s bigint
1	2020-05-04	-1
2	2020-05-02	0
3	2020-05-03	20
4	2020-05-01	2

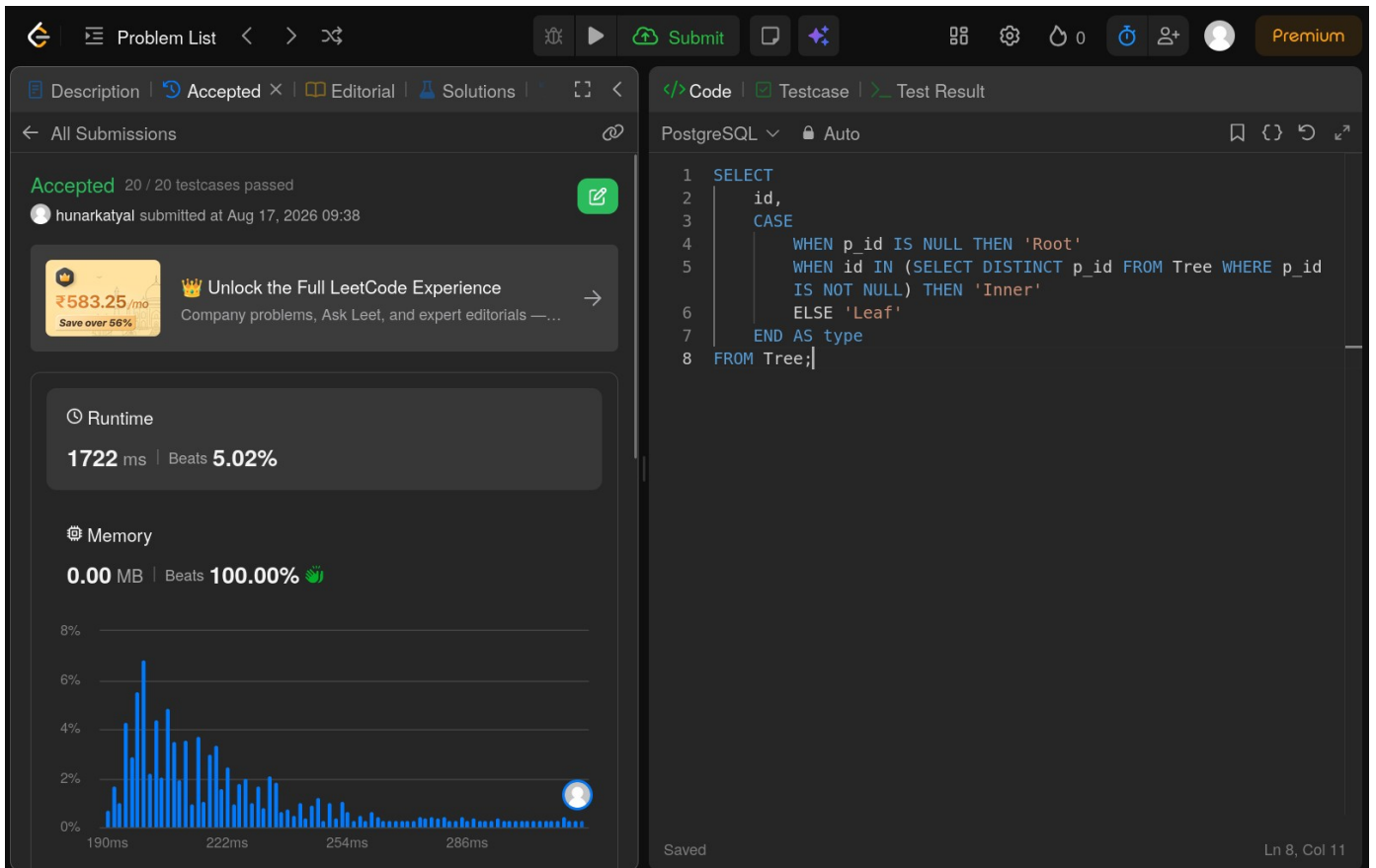
EXPERIMENT 6

4

Question: 6 (ii) Determine the type of Node

SQL Query:

```
SELECT DISTINCT t.id, (  
    CASE  
        WHEN t.p_id IS NULL THEN 'Root'  
        WHEN l.id IS NULL THEN 'Leaf'  
        ELSE 'Inner'  
    END  
)as type  
FROM Tree as t  
LEFT JOIN Tree as l  
ON t.id = l.p_id
```



The screenshot displays a LeetCode submission for a SQL problem. The top navigation bar includes links for Problem List, Submit, and other user options. The main content area is divided into two panels. The left panel shows the submission status as 'Accepted' with 20/20 testcases passed, submitted by 'hunarkatyal' on Aug 17, 2026. It also features a promotional banner for LeetCode's 'Unlock the Full LeetCode Experience' and performance metrics: Runtime 1722 ms (Beats 5.02%) and Memory 0.00 MB (Beats 100.00%). A bar chart at the bottom of the left panel shows the distribution of runtime times. The right panel displays the SQL query used for the submission, which is a PostgreSQL query to determine the type of each node in a tree structure. The query uses a CASE statement to classify nodes as 'Root', 'Leaf', or 'Inner' based on their parent and child relationships.

```
1 SELECT  
2     id,  
3     CASE  
4         WHEN p_id IS NULL THEN 'Root'  
5         WHEN id IN (SELECT DISTINCT p_id FROM Tree WHERE p_id  
6                     IS NOT NULL) THEN 'Inner'  
7         ELSE 'Leaf'  
8     END AS type  
9 FROM Tree;
```

PostgreSQL Auto

Saved Ln 8, Col 11

EXPERIMENT 6

4

Learning Outcome

After completing this experiment, the learner will be able to:

1. **Understand and implement correlated subqueries** to solve SQL problems involving row-wise comparisons.
2. **Apply aggregate functions** such as COUNT() and DISTINCT to retrieve top-N records within groups.
3. **Use joins and filtering techniques** to combine data from multiple tables and generate meaningful reports.
4. **Analyze and solve complex SQL queries** involving grouped data, ranking logic, and handling duplicate values effectively.